Demonstration of Space Complexity Using C

Name: ARITRA TALUKDAR
Roll No: CSB24034

Purpose

This report demonstrates constant, linear, and quadratic auxiliary space complexities using simple C programs. Only additional memory allocated by the algorithms is considered.

Algorithms Demonstrated

- **O(1) Space**: Finding the maximum element in an array using scalar variables.
- **O(n) Space**: Reversing an array using an auxiliary array.
- **O(n^2) Space**: Creating an adjacency matrix for a graph.

Source Code

```c
#include <stdio.h>
#include <stdlib.h>

/* ------------------------------------------
   CONSTANT SPACE ALGORITHM  -> O(1)
   Find maximum element in an array, no extra space is used.
------------------------------------------ */
int constantSpaceMax(int arr[], int n, size_t *auxMem)
{
    int max = arr[0];

    *auxMem = 0; // no auxiliary memory

    for (int i = 1; i < n; i++)
        if (arr[i] > max)
            max = arr[i];

    return max;
```

```
19 }
20
21 /* ------------------------------------------
22    LINEAR SPACE ALGORITHM  -> O(n)
23    Reverse array using extra array, that extra array is the
         auxiliary space used.
24 ------------------------------------------ */
25 void linearSpaceReverse(int arr[], int n, size_t *auxMem)
26 {
27     int *rev = (int *)malloc(n * sizeof(int));
28     if (!rev)
29         return;
30
31     *auxMem = n * sizeof(int); // auxiliary array
32
33     for (int i = 0; i < n; i++)
34         rev[i] = arr[n - i - 1];
35
36     free(rev);
37 }
38
39 /* ------------------------------------------
40    QUADRATIC SPACE ALGORITHM  -> O(n^2)
41    Graph adjacency matrix.The matrix itself is the auxiliary
         space used,
42     which grows quadratically with n.
43 ------------------------------------------ */
44 void quadraticSpaceGraph(int n, size_t *auxMem)
45 {
46     int **adj = (int **)malloc(n * sizeof(int *));
47     if (!adj)
48         return;
49
50     for (int i = 0; i < n; i++)
51         adj[i] = (int *)malloc(n * sizeof(int));
52
53     *auxMem = (n * sizeof(int *)) + (n * n * sizeof(int));
54
55     /* Initialize matrix */
56     for (int i = 0; i < n; i++)
57         for (int j = 0; j < n; j++)
```

```
            adj[i][j] = (i == j) ? 0 : 1;

    for (int i = 0; i < n; i++)
        free(adj[i]);
    free(adj);
}

int main()
{
    int n;
    size_t auxMem;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    int *arr = (int *)malloc(n * sizeof(int));
    for (int i = 0; i < n; i++)
        arr[i] = i + 1;

    /* ---------- CONSTANT SPACE ---------- */
    printf("\n===== CONSTANT SPACE =====\n");
    printf("Maximum Element: %d\n",
           constantSpaceMax(arr, n, &auxMem));
    printf("Auxiliary Memory Used: %zu bytes\n", auxMem);
    printf("Space Complexity: O(1)\n");

    /* ---------- LINEAR SPACE ---------- */
    printf("\n===== LINEAR SPACE =====\n");
    linearSpaceReverse(arr, n, &auxMem);
    printf("Auxiliary Memory Used: %zu bytes\n", auxMem);
    printf("Space Complexity: O(n)\n");

    /* ---------- QUADRATIC SPACE ---------- */
    printf("\n===== QUADRATIC SPACE =====\n");
    quadraticSpaceGraph(n, &auxMem);
    printf("Auxiliary Memory Used: %zu bytes\n", auxMem);
    printf("Space Complexity: O(n^2)\n");

    free(arr);
    return 0;
}
```

Program Output

The following screenshots show the combined output of all three algorithms executed in a single program run.

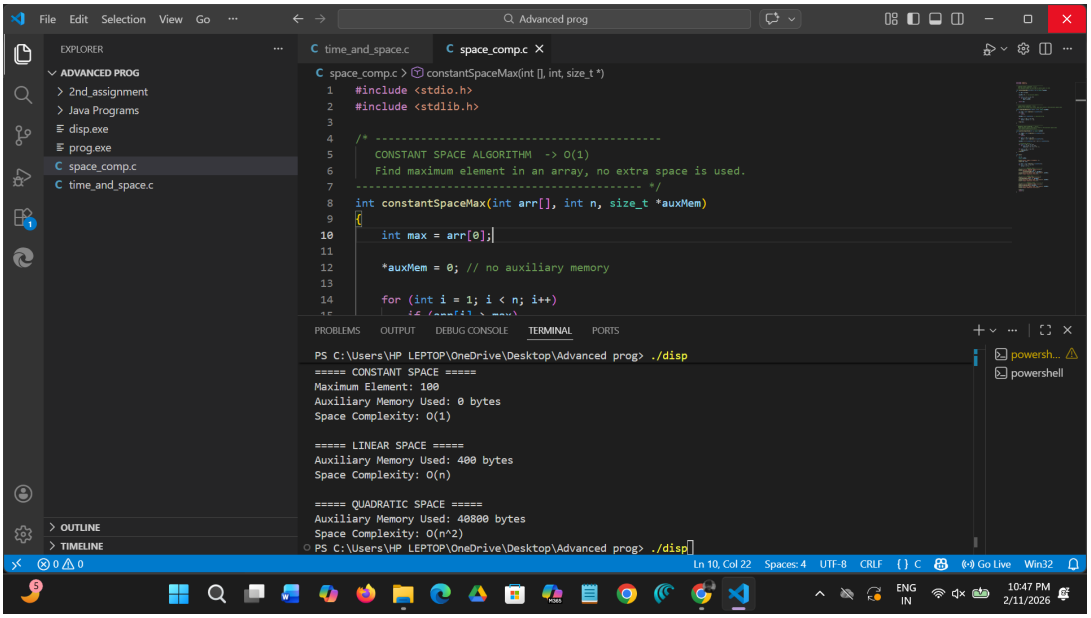

Figure 1: 1st sample output

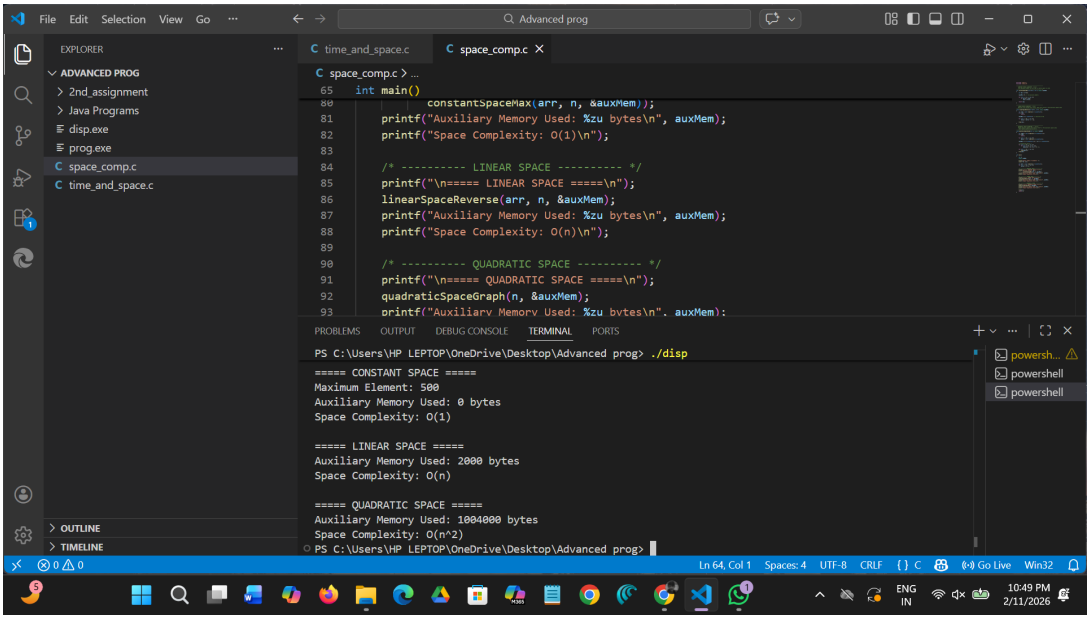

Figure 2: 2nd sample output

Conclusion

The program demonstrates how auxiliary space grows with different algorithm designs. Constant space algorithms use only scalar variables, linear space algorithms allocate

one additional array, and quadratic space algorithms require two-dimensional memory allocation.